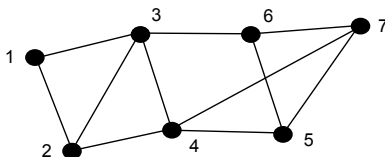


Introducción a la teoría de grafos

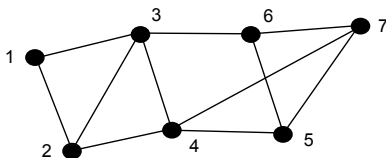
Programación III - UNGS

Definiciones



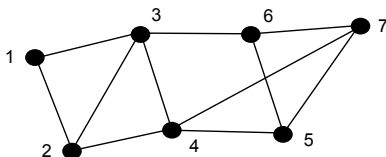
- **Definición.** Un **grafo** es un par $G = (V, E)$ tal que ...

Definiciones



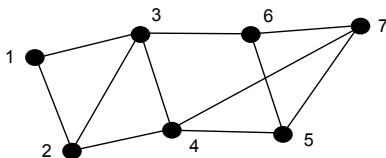
- **Definición.** Un **grafo** es un par $G = (V, E)$ tal que ...
 1. V es un conjunto finito (llamado el conjunto de **vértices** de G), y

Definiciones



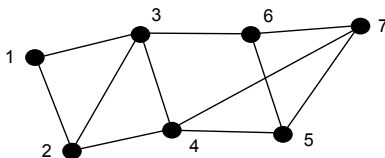
- **Definición.** Un **grafo** es un par $G = (V, E)$ tal que ...
 1. V es un conjunto finito (llamado el conjunto de **vértices** de G), y
 2. $E \subseteq \{ij \in V \times V : i \neq j\}$ es un conjunto de **aristas**.

Definiciones



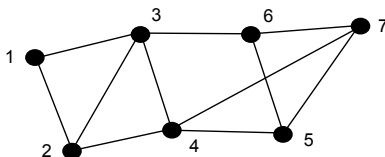
- Definición.** Un **grafo** es un par $G = (V, E)$ tal que ...
 - V es un conjunto finito (llamado el conjunto de **vértices** de G), y
 - $E \subseteq \{ij \in V \times V : i \neq j\}$ es un conjunto de **aristas**.
- En este ejemplo, $V = \{1, \dots, 7\}$ y $E = \{12, 13, 23, 24, 34, 36, 45, 47, 56, 57, 67\}$.

Definiciones



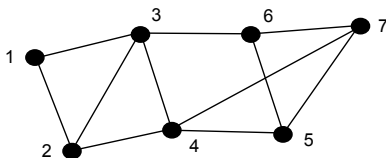
- El orden de los **extremos** de una arista no es importante. Nos referimos indistintamente a la arista 34 como 43 o bien $\{3, 4\}$.

Definiciones



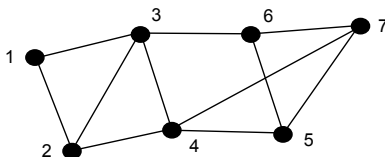
- El orden de los **extremos** de una arista no es importante. Nos referimos indistintamente a la arista 34 como 43 o bien $\{3, 4\}$.
- **Definiciones:**

Definiciones



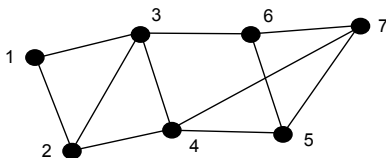
- El orden de los **extremos** de una arista no es importante. Nos referimos indistintamente a la arista 34 como 43 o bien $\{3, 4\}$.
- Definiciones:**
 - Si $ij \in E$, decimos que los vértices i y j son **vecinos**.

Definiciones



- El orden de los **extremos** de una arista no es importante. Nos referimos indistintamente a la arista 34 como 43 o bien $\{3, 4\}$.
- Definiciones:**
 - Si $ij \in E$, decimos que los vértices i y j son **vecinos**.
 - Dado $i \in V$, definimos el **vecindario** de i como $N(i) = \{j \in V : ij \in E\}$.

Definiciones



- El orden de los **extremos** de una arista no es importante. Nos referimos indistintamente a la arista 34 como 43 o bien $\{3, 4\}$.
- Definiciones:**
 - Si $ij \in E$, decimos que los vértices i y j son **vecinos**.
 - Dado $i \in V$, definimos el **vecindario** de i como $N(i) = \{j \in V : ij \in E\}$.
 - El **grado** de un vértice $i \in V$ es $d(i) = |N(i)|$.

Motivación

- Representamos con grafos relaciones **simétricas** entre entidades.

Motivación

- Representamos con grafos relaciones **simétricas** entre entidades.
 1. Rutas entre un grupo de ciudades (los vértices representan las ciudades).

Motivación

- Representamos con grafos relaciones **simétricas** entre entidades.
 1. Rutas entre un grupo de ciudades (los vértices representan las ciudades).
 2. Mapas de ciudades (los vértices representan las esquinas!).

Motivación

- Representamos con grafos relaciones **simétricas** entre entidades.
 1. Rutas entre un grupo de ciudades (los vértices representan las ciudades).
 2. Mapas de ciudades (los vértices representan las esquinas!).
 3. Relaciones de amistad en una red social (los vértices representan las personas).

Motivación

- Representamos con grafos relaciones **simétricas** entre entidades.
 1. Rutas entre un grupo de ciudades (los vértices representan las ciudades).
 2. Mapas de ciudades (los vértices representan las esquinas!).
 3. Relaciones de amistad en una red social (los vértices representan las personas).
 4. Cursos a dictar y superposiciones entre cursos (dos vértices son vecinos si sus cursos se solapan).

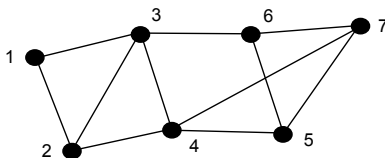
Motivación

- Representamos con grafos relaciones **simétricas** entre entidades.
 1. Rutas entre un grupo de ciudades (los vértices representan las ciudades).
 2. Mapas de ciudades (los vértices representan las esquinas!).
 3. Relaciones de amistad en una red social (los vértices representan las personas).
 4. Cursos a dictar y superposiciones entre cursos (dos vértices son vecinos si sus cursos se solapan).
 5. Antenas en una red de telefonía celular (dos vértices son vecinos si sus antenas tienen áreas de cobertura superpuestas).

Motivación

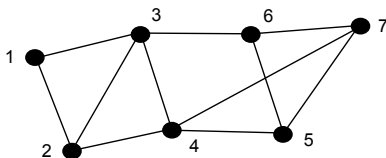
- Representamos con grafos relaciones **simétricas** entre entidades.
 1. Rutas entre un grupo de ciudades (los vértices representan las ciudades).
 2. Mapas de ciudades (los vértices representan las esquinas!).
 3. Relaciones de amistad en una red social (los vértices representan las personas).
 4. Cursos a dictar y superposiciones entre cursos (dos vértices son vecinos si sus cursos se solapan).
 5. Antenas en una red de telefonía celular (dos vértices son vecinos si sus antenas tienen áreas de cobertura superpuestas).
 6. Etc.

Definiciones



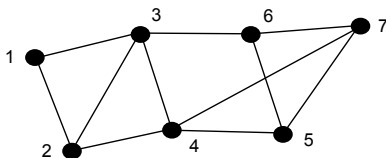
- Un **camino** entre dos vértices i y j es una secuencia de aristas desde i hasta j .

Definiciones



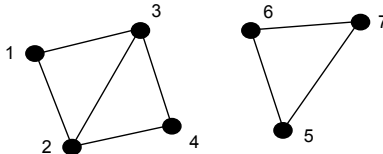
- Un **camino** entre dos vértices i y j es una secuencia de aristas desde i hasta j .
- La **distancia** entre dos vértices es la cantidad de aristas del camino más corto entre ellos.

Definiciones



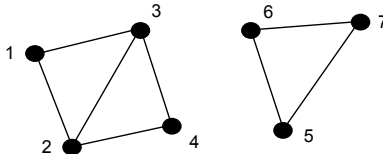
- Un **camino** entre dos vértices i y j es una secuencia de aristas desde i hasta j .
- La **distancia** entre dos vértices es la cantidad de aristas del camino más corto entre ellos.
- Un grafo es **conexo** si existe un camino entre todo par de vértices.

Definiciones



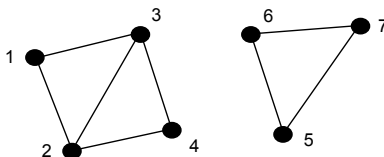
- Una **componente conexa** es un subconjunto de vértices conexo, maximal con esta propiedad.

Definiciones



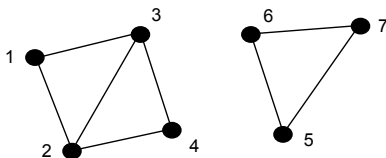
- Una **componente conexa** es un subconjunto de vértices conexo, maximal con esta propiedad.
 1. Un grafo conexo tiene exactamente una componente conexa.

Definiciones



- Una **componente conexa** es un subconjunto de vértices conexo, maximal con esta propiedad.
 1. Un grafo conexo tiene exactamente una componente conexa.
 2. Si i y j están en componentes conexas distintas, decimos que hay una distancia infinita entre ellos.

Definiciones



- Una **componente conexa** es un subconjunto de vértices conexo, maximal con esta propiedad.
 - Un grafo conexo tiene exactamente una componente conexa.
 - Si i y j están en componentes conexas distintas, decimos que hay una distancia infinita entre ellos.
- Un **vértice aislado** es un vértice i con $d(i) = 0$ (que conforma una componente conexa de tamaño 1).

Representación de grafos

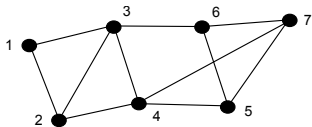
- Nos interesa programar algoritmos que trabajen sobre grafos. Para esto, debemos **representar** adecuadamente grafos en Java (¿cómo lo hacemos?).

Representación de grafos

- Nos interesa programar algoritmos que trabajen sobre grafos. Para esto, debemos **representar** adecuadamente grafos en Java (¿cómo lo hacemos?).
- **Definición.** La **matriz de adyacencia** de un grafo G es una matriz $A = (a_{ij}) \in \mathbb{R}^{|V| \times |V|}$ tal que $a_{ij} = 1$ si $ij \in E$ y $a_{ij} = 0$ en caso contrario.

Representación de grafos

- Nos interesa programar algoritmos que trabajen sobre grafos. Para esto, debemos **representar** adecuadamente grafos en Java (¿cómo lo hacemos?).
- Definición.** La **matriz de adyacencia** de un grafo G es una matriz $A = (a_{ij}) \in \mathbb{R}^{|V| \times |V|}$ tal que $a_{ij} = 1$ si $ij \in E$ y $a_{ij} = 0$ en caso contrario.



$$A = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 \end{pmatrix}$$

Representación de grafos

```

1  class Grafo
2  {
3      private int _vertices;
4      private boolean[][] _adj;
5
6      // Constructor
7      public Grafo(int n)
8      {
9          _vertices = n;
10         _adj = new boolean[n][n];
11     }
12
13     ...
14 }
```

Representación de grafos

```

1      ...
2
3      // Getters y setters de aristas
4      public void agregarArista(int i, int j)
5      {
6          _adj[i][j] = _adj[j][i] = true;
7      }
8      public void eliminarArista(int i, int j)
9      {
10         _adj[i][j] = _adj[j][i] = false;
11     }
12     public boolean existeArista(int i, int j)
13     {
14         return _adj[i][j];
15     }
16
17     ...

```

Representación de grafos

- Ventajas de esta representación:

Representación de grafos

- Ventajas de esta representación:
 1. Agregar una arista:

Representación de grafos

- Ventajas de esta representación:
 1. Agregar una arista: $O(1)$.

Representación de grafos

- Ventajas de esta representación:
 1. Agregar una arista: $O(1)$.
 2. Eliminar una arista:

Representación de grafos

- Ventajas de esta representación:
 1. Agregar una arista: $O(1)$.
 2. Eliminar una arista: $O(1)$.

Representación de grafos

- Ventajas de esta representación:
 1. Agregar una arista: $O(1)$.
 2. Eliminar una arista: $O(1)$.
 3. Consultar si existe arista:

Representación de grafos

- Ventajas de esta representación:
 1. Agregar una arista: $O(1)$.
 2. Eliminar una arista: $O(1)$.
 3. Consultar si existe arista: $O(1)$.

Representación de grafos

- Ventajas de esta representación:
 1. Agregar una arista: $O(1)$.
 2. Eliminar una arista: $O(1)$.
 3. Consultar si existe arista: $O(1)$.
- Desventajas de esta representación:

Representación de grafos

- Ventajas de esta representación:
 1. Agregar una arista: $O(1)$.
 2. Eliminar una arista: $O(1)$.
 3. Consultar si existe arista: $O(1)$.
- Desventajas de esta representación:
 1. Agregar un vértice:

Representación de grafos

- Ventajas de esta representación:
 1. Agregar una arista: $O(1)$.
 2. Eliminar una arista: $O(1)$.
 3. Consultar si existe arista: $O(1)$.
- Desventajas de esta representación:
 1. Agregar un vértice: $O(n^2)$.

Representación de grafos

- Ventajas de esta representación:
 1. Agregar una arista: $O(1)$.
 2. Eliminar una arista: $O(1)$.
 3. Consultar si existe arista: $O(1)$.
- Desventajas de esta representación:
 1. Agregar un vértice: $O(n^2)$.
 2. Eliminar un vértice:

Representación de grafos

- Ventajas de esta representación:
 1. Agregar una arista: $O(1)$.
 2. Eliminar una arista: $O(1)$.
 3. Consultar si existe arista: $O(1)$.
- Desventajas de esta representación:
 1. Agregar un vértice: $O(n^2)$.
 2. Eliminar un vértice: $O(n^2)$.

Representación de grafos

- Ventajas de esta representación:
 1. Agregar una arista: $O(1)$.
 2. Eliminar una arista: $O(1)$.
 3. Consultar si existe arista: $O(1)$.
- Desventajas de esta representación:
 1. Agregar un vértice: $O(n^2)$.
 2. Eliminar un vértice: $O(n^2)$.
 3. Obtener todos los vecinos de un vértice:

Representación de grafos

- Ventajas de esta representación:
 1. Agregar una arista: $O(1)$.
 2. Eliminar una arista: $O(1)$.
 3. Consultar si existe arista: $O(1)$.
- Desventajas de esta representación:
 1. Agregar un vértice: $O(n^2)$.
 2. Eliminar un vértice: $O(n^2)$.
 3. Obtener todos los vecinos de un vértice: $O(n)$.

Representación de grafos

- **Definición.** La **matriz de incidencia** de un grafo G es una matriz $M = (m_{ij}) \in \mathbb{R}^{|V| \times |E|}$ tal que $m_{ie} = 1$ si el vértice i es uno de los extremos de la arista e , y $m_{ie} = 0$ en caso contrario.

Representación de grafos

- **Definición.** La **matriz de incidencia** de un grafo G es una matriz $M = (m_{ij}) \in \mathbb{R}^{|V| \times |E|}$ tal que $m_{ie} = 1$ si el vértice i es uno de los extremos de la arista e , y $m_{ie} = 0$ en caso contrario.
- En nuestro ejemplo, $E = \{12, 13, 23, 24, 34, 36, 45, 47, 56, 57, 67\}$.

$$M = \begin{pmatrix} 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 \end{pmatrix}$$

Representación de grafos

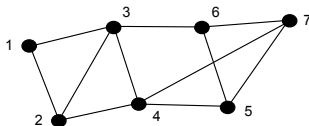
- **Definición.** La **matriz de incidencia** de un grafo G es una matriz $M = (m_{ij}) \in \mathbb{R}^{|V| \times |E|}$ tal que $m_{ie} = 1$ si el vértice i es uno de los extremos de la arista e , y $m_{ie} = 0$ en caso contrario.
- En nuestro ejemplo, $E = \{12, 13, 23, 24, 34, 36, 45, 47, 56, 57, 67\}$.

$$M = \begin{pmatrix} 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 \end{pmatrix}$$

- ¿Es una representación conveniente?

Representación de grafos

- Una tercera opción es por medio de **listas de vecinos** asociadas con cada vértice:



1 → 2, 3
 2 → 1, 3, 4
 3 → 1, 2, 4, 6
 4 → 2, 3, 5, 7
 5 → 4, 6, 7
 6 → 3, 5, 7
 7 → 4, 5, 6

Representación de grafos

```

1  class Grafo
2  {
3      private ArrayList<HashSet<Integer>> _vecinos;
4
5      // Constructor
6      public Grafo(int n)
7      {
8          _vecinos = new ArrayList<HashSet<Integer>>();
9          for(int i=0; i<n; ++i)
10             _vecinos.add(new HashSet<Integer>());
11     }
12
13     ...
14 }
```

Representación de grafos

```

1
2 // Getters y setters de aristas
3 public void agregarArista(int i, int j)
4 {
5     _vecinos.get(i).add(j);
6     _vecinos.get(j).add(i);
7 }
8 public void eliminarArista(int i, int j)
9 {
10    _vecinos.get(i).remove(j);
11    _vecinos.get(j).remove(i);
12 }
13 public boolean existeArista(int i, int j)
14 {
15    return _vecinos.get(i).contains(j);
16 }
17
18

```

Representación de grafos

- Ventajas de esta representación:

Representación de grafos

- Ventajas de esta representación:
 1. Obtener todos los vecinos de un vértice:

Representación de grafos

- Ventajas de esta representación:
 1. Obtener todos los vecinos de un vértice: $O(1)$.

Representación de grafos

- Ventajas de esta representación:
 1. Obtener todos los vecinos de un vértice: $O(1)$.
 2. Agregar un vértice:

Representación de grafos

- Ventajas de esta representación:
 1. Obtener todos los vecinos de un vértice: $O(1)$.
 2. Agregar un vértice: $O(n)$ en el peor caso,

Representación de grafos

- Ventajas de esta representación:
 1. Obtener todos los vecinos de un vértice: $O(1)$.
 2. Agregar un vértice: $O(n)$ en el peor caso, pero $O(1)$ amortizado.

Representación de grafos

- Ventajas de esta representación:
 1. Obtener todos los vecinos de un vértice: $O(1)$.
 2. Agregar un vértice: $O(n)$ en el peor caso, pero $O(1)$ amortizado.
 3. Agregar una arista:

Representación de grafos

- Ventajas de esta representación:
 1. Obtener todos los vecinos de un vértice: $O(1)$.
 2. Agregar un vértice: $O(n)$ en el peor caso, pero $O(1)$ amortizado.
 3. Agregar una arista: $O(1)$ promedio.

Representación de grafos

- Ventajas de esta representación:
 1. Obtener todos los vecinos de un vértice: $O(1)$.
 2. Agregar un vértice: $O(n)$ en el peor caso, pero $O(1)$ amortizado.
 3. Agregar una arista: $O(1)$ promedio.
 4. Eliminar una arista:

Representación de grafos

- Ventajas de esta representación:
 1. Obtener todos los vecinos de un vértice: $O(1)$.
 2. Agregar un vértice: $O(n)$ en el peor caso, pero $O(1)$ amortizado.
 3. Agregar una arista: $O(1)$ promedio.
 4. Eliminar una arista: $O(1)$ promedio.

Representación de grafos

- Ventajas de esta representación:
 1. Obtener todos los vecinos de un vértice: $O(1)$.
 2. Agregar un vértice: $O(n)$ en el peor caso, pero $O(1)$ amortizado.
 3. Agregar una arista: $O(1)$ promedio.
 4. Eliminar una arista: $O(1)$ promedio.
 5. Consultar si existe arista:

Representación de grafos

- Ventajas de esta representación:
 1. Obtener todos los vecinos de un vértice: $O(1)$.
 2. Agregar un vértice: $O(n)$ en el peor caso, pero $O(1)$ amortizado.
 3. Agregar una arista: $O(1)$ promedio.
 4. Eliminar una arista: $O(1)$ promedio.
 5. Consultar si existe arista: $O(1)$ promedio.

Representación de grafos

- Ventajas de esta representación:
 1. Obtener todos los vecinos de un vértice: $O(1)$.
 2. Agregar un vértice: $O(n)$ en el peor caso, pero $O(1)$ amortizado.
 3. Agregar una arista: $O(1)$ promedio.
 4. Eliminar una arista: $O(1)$ promedio.
 5. Consultar si existe arista: $O(1)$ promedio.
- Desventajas de esta representación:

Representación de grafos

- Ventajas de esta representación:
 1. Obtener todos los vecinos de un vértice: $O(1)$.
 2. Agregar un vértice: $O(n)$ en el peor caso, pero $O(1)$ amortizado.
 3. Agregar una arista: $O(1)$ promedio.
 4. Eliminar una arista: $O(1)$ promedio.
 5. Consultar si existe arista: $O(1)$ promedio.
- Desventajas de esta representación:
 1. Eliminar un vértice:

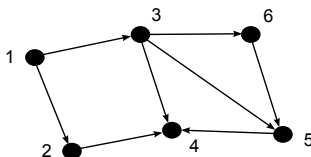
Representación de grafos

- Ventajas de esta representación:
 1. Obtener todos los vecinos de un vértice: $O(1)$.
 2. Agregar un vértice: $O(n)$ en el peor caso, pero $O(1)$ amortizado.
 3. Agregar una arista: $O(1)$ promedio.
 4. Eliminar una arista: $O(1)$ promedio.
 5. Consultar si existe arista: $O(1)$ promedio.
- Desventajas de esta representación:
 1. Eliminar un vértice: $O(n)$.

Representación de grafos

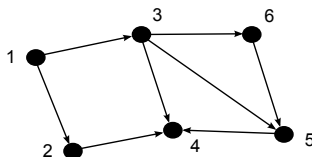
- Ventajas de esta representación:
 1. Obtener todos los vecinos de un vértice: $O(1)$.
 2. Agregar un vértice: $O(n)$ en el peor caso, pero $O(1)$ amortizado.
 3. Agregar una arista: $O(1)$ promedio.
 4. Eliminar una arista: $O(1)$ promedio.
 5. Consultar si existe arista: $O(1)$ promedio.
- Desventajas de esta representación:
 1. Eliminar un vértice: $O(n)$.
- Las operaciones sobre aristas son $O(\log n)$ en el peor caso si se usan **TreeSets** para representar los vecinos de cada vértice.

Grafos dirigidos



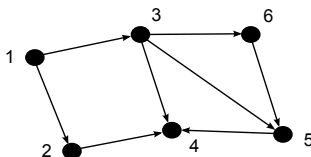
- **Definición.** Un **grafo dirigido** (o digrafo) es un par $D = (N, A)$ tal que ...

Grafos dirigidos



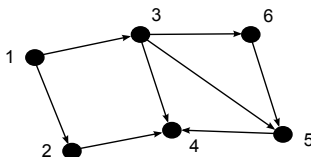
- **Definición.** Un **grafo dirigido** (o digrafo) es un par $D = (N, A)$ tal que ...
 1. N es un conjunto finito (llamado el conjunto de **nodos** de G), y

Grafos dirigidos



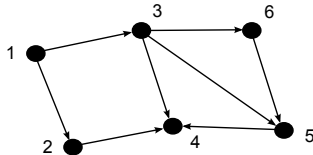
- **Definición.** Un **grafo dirigido** (o digrafo) es un par $D = (N, A)$ tal que ...
 1. N es un conjunto finito (llamado el conjunto de **nodos** de G), y
 2. $A \subseteq \{(i, j) \in V \times V : i \neq j\}$ es un conjunto de pares ordenados, llamados **arcos**.

Grafos dirigidos



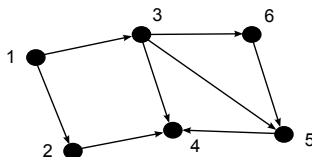
- Definición.** Un **grafo dirigido** (o digrafo) es un par $D = (N, A)$ tal que ...
 - N es un conjunto finito (llamado el conjunto de **nod**os de G), y
 - $A \subseteq \{(i, j) \in V \times V : i \neq j\}$ es un conjunto de pares ordenados, llamados **arcos**.
- A diferencia de los grafos (no dirigidos), ahora las aristas son **pares ordenados**.

Definiciones



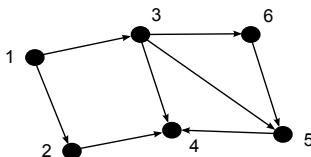
- Diferenciamos entre el **grado de entrada** y el **grado de salida** de cada nodo.

Definiciones



- Diferenciamos entre el **grado de entrada** y el **grado de salida** de cada nodo.
- $d^-(i) = |N^-(i)| = |\{j \in N : (j, i) \in A\}|$.

Definiciones

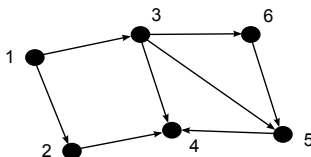


- Diferenciamos entre el **grado de entrada** y el **grado de salida** de cada nodo.

- $d^-(i) = |N^-(i)| = |\{j \in N : (j, i) \in A\}|.$

- $d^+(i) = |N^+(i)| = |\{j \in N : (i, j) \in A\}|.$

Definiciones



- Diferenciamos entre el **grado de entrada** y el **grado de salida** de cada nodo.
 - $d^-(i) = |N^-(i)| = |\{j \in N : (j, i) \in A\}|$.
 - $d^+(i) = |N^+(i)| = |\{j \in N : (i, j) \in A\}|$.
- Un camino entre dos nodos es una secuencia de arcos entre ellos, respetando el sentido de los arcos.